

# Basics of Python Programming

Core syntax, data types, control flow, functions and error handling

## **In this guide**

- Introduction
- 1. Setting Up and Running Python
- 2. Variables, Data Types and Operators
- 3. Control Flow: Conditions and Loops
- 4. Functions and Modules
- 5. Core Data Structures
- 6. Files, Errors and Exceptions
- Conclusion

# Introduction

---

Python is a high-level, general-purpose programming language known for readable syntax and a huge ecosystem of libraries. It is used for web development, automation, scientific computing, data analysis, and artificial intelligence. Its design philosophy favors clarity: code is indented rather than wrapped in braces, and most tasks can be expressed in only a few lines.

This guide walks through the fundamentals in the order most learners need them. Each section introduces a small set of ideas, shows a short example, and ends with key points you can use as a checklist. You do not need any prior programming experience, but you should be comfortable installing software and using a text editor.

## Why learn Python?

- **Readable:** the syntax reads close to plain English, which lowers the barrier to entry.
- **Versatile:** the same language works for scripts, websites, data pipelines and machine learning.
- **Well supported:** millions of developers, thousands of packages, and abundant tutorials.
- **Interactive:** you can test ideas instantly in a REPL or notebook.

## What you will learn

- 1 How to install Python and run your first program.
- 2 How to store and manipulate data with variables, types and operators.
- 3 How to make decisions and repeat actions with conditions and loops.
- 4 How to organize code with functions and modules.
- 5 How to use lists, tuples, dictionaries and sets.
- 6 How to read and write files and handle errors gracefully.

## How to use this guide

Type every example yourself rather than copy-pasting. Changing values, breaking the code on purpose, and reading the error messages is the fastest way to build intuition. Short practice tasks appear in the key points of each section.

**Tip:** Python 3 is the current version. Everything in this guide assumes Python 3.9 or newer.

# 1. Setting Up and Running Python

Before writing code you need an interpreter, which is the program that reads your Python source and executes it. Download it from [python.org](https://python.org), or install it through a package manager such as Homebrew (macOS), apt (Linux) or winget (Windows). After installation, open a terminal and run `python --version` to confirm that it works.

## Three ways to run code

- **The REPL:** type `python` in a terminal to get an interactive prompt. Ideal for quick experiments.
- **Script files:** save code in a file ending in `.py` and run it with `python script.py`.
- **Notebooks:** Jupyter notebooks mix code, output and notes; popular for data work and teaching.

Your first program prints a message to the screen. The `print()` function is the simplest way to see what your program is doing, and you will use it constantly while learning.

```
# hello.py
name = "Ada"
print("Hello,", name)
print(f"Welcome to Python, {name}!") # f-string formatting
```

## Virtual environments

As soon as you install third-party packages you should isolate them per project. A virtual environment is a private folder that holds a project's packages so different projects cannot conflict.

```
python -m venv .venv          # create the environment
source .venv/bin/activate     # macOS/Linux (Windows: .venv\Scripts\activate)
pip install requests          # install a package into it
```

## Key points

- Python code runs from top to bottom, one statement per line.
- Indentation is part of the syntax; use four spaces per level.
- Comments start with `#` and are ignored by the interpreter.
- Use `pip` to install packages and virtual environments to keep projects separate.

## 2. Variables, Data Types and Operators

A variable is a name that refers to a value. You create one by assigning with `=`; there is no need to declare a type first. Python is dynamically typed, meaning that the type belongs to the value, not the variable, so the same name can later refer to a different kind of value.

### Built-in types

| Type                  | Example                                | Purpose                   |
|-----------------------|----------------------------------------|---------------------------|
| <code>int</code>      | <code>42</code>                        | Whole numbers of any size |
| <code>float</code>    | <code>3.14</code>                      | Decimal numbers           |
| <code>str</code>      | <code>"hello"</code>                   | Text                      |
| <code>bool</code>     | <code>True</code> , <code>False</code> | Logical values            |
| <code>NoneType</code> | <code>None</code>                      | Represents 'no value'     |

Use `type(x)` to inspect a value and `int()`, `float()` or `str()` to convert between types. Conversions can fail: `int("abc")` raises a `ValueError`.

```
age = 30
price = 19.99
title = "Data Book"
in_stock = True

print(type(price))          # <class 'float'>
total = price * 3
print(f"Total: {total:.2f}") # Total: 59.97
print(int("12") + 8)        # 20
```

### Operators

- **Arithmetic:** `+`, `-`, `*`, `/`, `//`, `%`, `**` (`//` is floor division, `%` is remainder, `**` is power).
- **Comparison:** `==`, `!=`, `<`, `>`, `<=`, `>=` produce a boolean.
- **Logical:** `and`, `or`, `not` combine booleans.
- **Membership:** `in` and `not in` test whether a value is inside a collection or string.

Strings support slicing and many helpful methods. Indexing starts at zero, and negative indexes count from the end.

```
s = "python"
print(s[0], s[-1], s[1:4]) # p n yth
print(s.upper(), len(s))  # PYTHON 6
print(" hi ".strip())     # hi
```

### Key points

- Choose descriptive names in `snake_case`, such as `total_price`.

- `/` always returns a float; use `//` for integer division.
- Strings are immutable: methods return new strings rather than changing the original.
- f-strings (`f"{value}"`) are the clearest way to build formatted text.

## 3. Control Flow: Conditions and Loops

Programs become useful when they can make decisions and repeat work. Conditions choose between paths, and loops run a block of code many times. Both rely on indentation to define which lines belong to the block.

### Conditional statements

An `if` statement runs a block only when its condition is true. Add `elif` for extra cases and `else` for everything that remains. Python treats empty strings, empty collections, `0` and `None` as false, and most other values as true.

```
score = 78
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
else:
    grade = "C"
print(grade)    # B
```

### for loops

A `for` loop visits each item in a sequence. Use `range(n)` to repeat something a fixed number of times, and `enumerate()` when you need both the position and the value.

```
for i in range(3):
    print("step", i)           # step 0, step 1, step 2

fruits = ["apple", "banana", "cherry"]
for idx, fruit in enumerate(fruits, start=1):
    print(idx, fruit)
```

### while loops and loop control

A `while` loop repeats as long as its condition remains true, so make sure something inside the loop eventually makes the condition false. Two keywords give finer control: `break` exits the loop immediately and `continue` skips to the next iteration.

```
total = 0
n = 1
while total < 20:
    total += n
    n += 1
print(total, n)    # 21 7
```

## Comprehensions

A list comprehension builds a new list in one readable line. Use it for simple transformations and filters; switch to a normal loop if it becomes hard to read.

```
squares = [x * x for x in range(6)]          # [0, 1, 4, 9, 16, 25]
evens = [x for x in range(10) if x % 2 == 0]  # [0, 2, 4, 6, 8]
```

## Key points

- Conditions end with a colon and the following block is indented.
- Use **for** when you know what you are iterating over; use **while** when you loop until a condition changes.
- Avoid infinite loops by updating the loop condition inside the body.
- Practice: write a loop that prints the multiplication table for 7.

## 4. Functions and Modules

A function is a named, reusable block of code. Functions let you avoid repetition, give ideas descriptive names, and make programs easier to test. You define one with **def**, list its parameters, and send back a result with **return**. A function without a **return** statement gives back **None**.

```
def area(width, height=1):
    """Return the area of a rectangle."""
    return width * height

print(area(4, 5))          # 20
print(area(4))              # 4   (default height)
print(area(height=2, width=3)) # 6   (keyword arguments)
```

## Parameters and return values

- **Positional arguments** are matched by order; **keyword arguments** by name.
- **Default values** make parameters optional; they must come after required ones.
- **\*args** collects extra positional arguments and **\*\*kwargs** collects extra keyword arguments.
- A function can return several values at once as a tuple: **return low, high**.

## Scope

Variables created inside a function are local: they exist only while it runs. A function can read global variables, but changing them requires the **global** keyword, which is best avoided. Prefer passing values in through parameters and getting results out through **return**.

## Lambda functions

A lambda is a tiny anonymous function written on one line. It is often used as a sort key or inside **map** and **filter**.

```
people = [("Sam", 31), ("Lea", 25), ("Omar", 40)]
people.sort(key=lambda p: p[1])
print(people[0])          # ('Lea', 25)
```

## Modules and packages

A module is simply a `.py` file whose functions can be imported by other files. The standard library ships with many useful modules, such as `math`, `random`, `datetime`, `json` and `pathlib`.

```
import math
from random import randint

print(math.sqrt(49))      # 7.0
print(randint(1, 6))      # a random die roll
```

## Key points

- Give each function one clear job and a docstring that explains it.
- Prefer returning values over printing inside functions.
- Never use a mutable default such as `def f(items=[])`; use `None` and create the list inside.
- Split larger programs into modules to keep files short and focused.

## 5. Core Data Structures

Python includes four built-in collection types that cover most everyday needs. Choosing the right one makes code simpler and faster.

| Structure | Syntax                 | Ordered            | Mutable | Best for                       |
|-----------|------------------------|--------------------|---------|--------------------------------|
| list      | <code>[1, 2, 3]</code> | Yes                | Yes     | Sequences that change          |
| tuple     | <code>(1, 2, 3)</code> | Yes                | No      | Fixed records, dict keys       |
| dict      | <code>{"a": 1}</code>  | Yes (by insertion) | Yes     | Lookup by key                  |
| set       | <code>{1, 2, 3}</code> | No                 | Yes     | Unique items, membership tests |

## Lists

Lists are the workhorse collection. You can add, remove, sort and slice them freely.

```
nums = [5, 2, 9]
nums.append(1)          # [5, 2, 9, 1]
nums.sort()             # [1, 2, 5, 9]
print(nums[1:3], len(nums), sum(nums))  # [2, 5] 4 17
```

## Dictionaries

A dictionary maps keys to values and offers very fast lookups. Use `.get()` to read a key safely with a fallback, and `.items()` to loop through both keys and values.

```
user = {"name": "Mia", "age": 28}
user["city"] = "Pune"
print(user.get("email", "not set"))    # not set
for key, value in user.items():
    print(key, "->", value)
```

## Tuples and sets

Tuples are like lists that cannot change, which makes them safe for fixed data such as coordinates. Sets store only unique values and support mathematical operations.

```
point = (3, 4)
x, y = point                # unpacking
tags = {"ai", "data", "ai"} # {'ai', 'data'}
print({1, 2, 3} & {2, 3, 4}) # intersection: {2, 3}
```

## Key points

- Use a list when order and change matter, a dict when you look things up by name.
- Use a set to remove duplicates or test membership quickly.
- Copying: `b = a` makes both names point to the same list; use `a.copy()` for a real copy.
- Practice: count how often each word appears in a sentence using a dictionary.

# 6. Files, Errors and Exceptions

Real programs read data from files and must cope with things going wrong. Python's `with` statement opens a file and guarantees it is closed afterwards, even if an error occurs.

## Reading and writing files

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("first line\n")
    f.write("second line\n")

with open("notes.txt", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

- `"r"` reads (default), `"w"` overwrites, `"a"` appends.
- Always specify `encoding="utf-8"` for text files to avoid platform surprises.
- `pathlib.Path` gives an object-oriented way to build and inspect file paths.

## Handling exceptions

When Python cannot continue, it raises an exception. An unhandled exception stops the program and prints a traceback. Wrap risky code in `try` and catch the specific exceptions you expect.

```
def read_number(text):
    try:
        return int(text)
    except ValueError:
        print("Not a valid number:", text)
        return None
    finally:
        print("checked", text)      # always runs

read_number("42")
read_number("forty")
```

## Common exceptions

| Exception                      | Typical cause                                                   |
|--------------------------------|-----------------------------------------------------------------|
| <code>ValueError</code>        | Right type, wrong value, such as <code>int("x")</code>          |
| <code>TypeError</code>         | Operation on an incompatible type, such as <code>"a" + 1</code> |
| <code>KeyError</code>          | Missing dictionary key                                          |
| <code>IndexError</code>        | List index out of range                                         |
| <code>FileNotFoundError</code> | Opening a file that does not exist                              |

## Debugging tips

- 1 Read the traceback from the bottom up; the last line names the error.
- 2 Print intermediate values or use the built-in debugger (`breakpoint()`).
- 3 Reduce the failing code to the smallest example that still breaks.

## Key points

- Use `with` for files so resources are released automatically.
- Catch specific exceptions; avoid a bare `except:` that hides bugs.
- Raise your own errors with `raise ValueError("message")` when input is invalid.

## Conclusion

---

You have now met the essential building blocks of Python: how to run code, store data in variables, control the flow of a program, package logic into functions, organize information in collections, and deal with files and errors. These ideas appear in nearly every Python program you will ever read or write, so time spent mastering them pays off quickly.

### Key takeaways

- Python favors readable, indented code and dynamic typing.
- Lists, tuples, dictionaries and sets cover most data-storage needs.
- Functions and modules keep programs organized and reusable.
- Exceptions are normal; handle the ones you expect and read tracebacks carefully.

### Suggested next steps

- 1 Build three small projects: a to-do list, a unit converter and a word counter.
- 2 Learn object-oriented programming: classes, methods and inheritance.
- 3 Explore the standard library (`collections`, `itertools`, `json`, `csv`).
- 4 Move on to libraries such as Pandas, NumPy or Flask depending on your goals.

**Remember:** Programming is a skill built by practice. Write a little code every day, and treat every error message as a hint rather than a failure.